Mini-Project: E-Vaporiser Image Classifier (Transfer Learning)

An extension of *Lab 3 — DNN Workflow Part 2*

The problem (MVP framing). An online classified-ads marketplace wants to **automatically flag listings whose photos show an e-vaporiser (vape)** so they can be blocked before going live. This notebook builds the model behind that feature: a binary image classifier that answers one question for any uploaded photo — **"is this a vape, yes or no?"**

What this notebook does, end-to-end:

1. **Data** — loads a real vape image dataset (Roboflow) as the *positive* class and everyday-object photos as the *negative* class, then splits into train/val/test.
2. **Augmentation** — applies and *justifies* image augmentations, and visualises them.
3. **Models** — compares **two backbones**:
   - **ResNet** from **torchvision** (a CNN)
   - **ViT** (Vision Transformer) from **Hugging Face** (a transformer)
4. **Strategies** — trains **each** backbone with **two strategies**:
   - **Full fine-tune** — update *all* layers
   - **Head-only** — freeze the backbone, train *only* the new classifier head
5. **Tracking** — logs every run to **Weights & Biases (wandb.ai)**.
6. **Comparison** — loss/accuracy curves, a test-accuracy table, and a confusion matrix per configuration.
7. **Selection** — picks the **two best configurations** and saves their weights for the seller-portal web app (separate deliverables).

> This is written in the same beginner-friendly, heavily-commented style as Lab 3. Run the cells **top to bottom**. A **GPU runtime is strongly recommended** (Colab: *Runtime → Change runtime type → GPU*).

Configuration matrix (4 training runs):

	Full fine-tune	Head-only
ResNet (torchvision)	`resnet_full`	`resnet_head`
ViT (Hugging Face)	`vit_full`	`vit_head`

🔁 What's new in v2 (post-review enhancements)

#	Change	
1	**Hard negatives** — negatives are now higher-res **Imagenette-320** *plus* vape-lookalike objects (pens, flashlights, phones, remotes, microphones) from **Open Images**	v
2	**Selection on validation only**	v
3	**Step 10b: ROC/PR curves + threshold tuning**	T
4	**Class-weighted loss**	v
5	**Security & reproducibility**	T

> **Expect accuracy to drop below v1's ~100%.** That is the point: the numbers now measure vape recognition, not photo-quality shortcuts, and the backbone/strategy comparison finally has something to say.

Step 0 — Install dependencies

We install everything up front. On Colab most of these are already present; the cell is safe to re-run.

- `torch`, `torchvision` — the ResNet backbone + training engine
- `transformers` — the Hugging Face ViT backbone
- `roboflow` — to download the vape dataset
- `wandb` — experiment tracking on wandb.ai
- `scikit-learn`, `seaborn`, `matplotlib` — metrics & plots
- `fiftyone` — keyless download of Open Images **hard negatives** (new in v2)
- `gradio` — used by the *web-app* deliverables (imported here only to pin the version)

```
# %%capture hides the long pip output; remove it if you want to see the logs.
#%%capture
!pip install -q torch torchvision transformers roboflow wandb scikit-learn seaborn matplotlib gradio fifty
```

Step 1 — Imports and device

We pick the GPU automatically if one is available. Everything else is standard PyTorch.

```
import os, copy, random, glob, shutil, time
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

import torchvision
from torchvision import transforms
from torchvision.models import resnet18, ResNet18_Weights

from transformers import ViTForImageClassification, ViTImageProcessor

from sklearn.metrics import (confusion_matrix, classification_report,
                             accuracy_score, precision_recall_fscore_support,
                             precision_score, recall_score,
                             roc_curve, precision_recall_curve, auc)

# Reproducibility: fix every seed we can so the split, shuffling and results are repeatable.
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)

def seed_worker(worker_id):
    # Each DataLoader worker gets its own RNG, derived from the (seeded) global one.
    ws = torch.initial_seed() % 2**32
    np.random.seed(ws); random.seed(ws)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)
if device.type != 'cuda':
    print('WARNING: no GPU detected. Training (especially the ViT) will be slow. '
          'On Colab: Runtime -> Change runtime type -> GPU.')
```

```
Using device: cuda
```

Step 2 — Log in to Weights & Biases (wandb.ai)

`wandb` records the loss/accuracy of every run to your online dashboard so the four configurations can be compared side-by-side.

1. Make a free account at https://wandb.ai and copy your API key from https://wandb.ai/authorize
2. Paste it when prompted below.

If you **don't** have a key, set `USE_WANDB = False` and the notebook still runs end-to-end (it just won't log online).

```
import wandb

USE_WANDB = False          # set to False to skip online logging entirely
WANDB_PROJECT = 'evaporiser-classifier'

if USE_WANDB:
    try:
        wandb.login()      # paste your key when prompted (or set WANDB_API_KEY env var)
        print('wandb login OK ->', WANDB_PROJECT)
    except Exception as e:
        print('wandb login failed, continuing without online logging:', e)
        USE_WANDB = False
else:
    print('wandb disabled by USE_WANDB = False')
```

```
    wandb disabled by USE_WANDB = False
```

Step 3 — Get the data

We build a **binary** dataset with two folders:

```
data/
  vape/       <- POSITIVE class: e-vaporiser photos (from Roboflow)
  not_vape/   <- NEGATIVE class: everyday objects + vape-lookalikes (Imagenette-320 + Open Images)
```

3a. Positives — the Roboflow vape dataset

We use the public `vape-dataset/vape-dataset` project on Roboflow Universe (https://universe.roboflow.com/vape-dataset/vape-dataset).

> **You need a free Roboflow API key.** Get it from your Roboflow account settings. **Never paste the key into the notebook** — store it as a Colab **Secret** named `ROBOFLOW_API_KEY` (Colab: 🔑 icon in the left sidebar), or export it as an environment variable; the config cell below falls back to a hidden prompt if neither is set. Many Universe projects are *object-detection* datasets. For our *whole-image* classifier we only need the images, so the code downloads the dataset and **treats every downloaded image as a `vape` example**.

3b. Negatives — everyday objects **+ hard negatives** (v2 change)

A classifier that only ever sees vapes would call *everything* a vape, so we need a contrasting class — and it must be a **fair** one. v1 used 160-px Imagenette scenes, which let the model "win" by detecting photo style/resolution instead of vapes (every config scored ~100%). v2 fixes both problems:

- **Base negatives — Imagenette-320.** The same 10 everyday categories, but at **320 px**, so image resolution is no longer a giveaway.
- **Hard negatives — Open Images (no key needed).** Objects that genuinely *look like* vapes in a phone photo: **pens, flashlights, mobile phones, remote controls, microphones**. If the model separates these from vapes, it has learned real vape features — not "small gadget = vape".

Roughly **half** of the negative folder is hard negatives; the rest is everyday scenes.

Configuration — edit these, then run the download cells

```
# ----- Roboflow API key — NEVER hardcode it in the notebook ---------------
# Priority: Colab Secret 'ROBOFLOW_API_KEY'  ->  environment variable  ->  hidden prompt.
ROBOFLOW_API_KEY = 'MSSc8gUEDH7KGSYwZpTK'
try:
    from google.colab import userdata          # Colab: key icon in the left sidebar -> add secret
    ROBOFLOW_API_KEY = userdata.get('ROBOFLOW_API_KEY')
except Exception:
    pass
if not ROBOFLOW_API_KEY:
    ROBOFLOW_API_KEY = os.environ.get('ROBOFLOW_API_KEY')
if not ROBOFLOW_API_KEY:
    from getpass import getpass
    ROBOFLOW_API_KEY = getpass('Paste your Roboflow API key (input is hidden): ')

# ----- EDIT ME ------------------------------------------------------------
RF_WORKSPACE     = 'vape-dataset'
RF_PROJECT       = 'vape-dataset'
RF_VERSION       = 1           # <-- set to the version shown on the dataset page
RF_EXPORT_FORMAT = 'folder'    # 'folder' (classification) usually works; if it errors,
                               # try 'coco' or 'voc' — we only read the image files anyway.

MAX_PER_CLASS     = 800        # cap images per class to keep training fast & balanced
HARD_NEG_FRACTION = 0.5        # up to this fraction of negatives are vape-lookalikes (v2)
# --------------------------------------------------------------------------

DATA_ROOT = '/content/data'    # final binary dataset lives here
VAPE_DIR     = os.path.join(DATA_ROOT, 'vape')
NOT_VAPE_DIR = os.path.join(DATA_ROOT, 'not_vape')
os.makedirs(VAPE_DIR, exist_ok=True)
os.makedirs(NOT_VAPE_DIR, exist_ok=True)

# staging folders for the two negative sources (composed into not_vape/ in step 3d)
```

```
IMAGENETTE_DIR = '/content/imagenette2-320'
HARD_NEG_DIR   = '/content/hard_negatives'
os.makedirs(HARD_NEG_DIR, exist_ok=True)

IMG_EXTS = ('.jpg', '.jpeg', '.png', '.bmp', '.webp')

def collect_images(src_root):
    # Recursively list every image file under a folder.
    files = []
    for dirpath, _, filenames in os.walk(src_root):
        for fn in filenames:
            if fn.lower().endswith(IMG_EXTS):
                files.append(os.path.join(dirpath, fn))
    return files

def copy_sample(files, dst_dir, max_n, prefix=''):
    # Copy up to max_n images into dst_dir with simple sequential names.
    # `prefix` keeps files from different sources from overwriting each other.
    random.shuffle(files)
    n = 0
    for src in files[:max_n]:
        try:
            Image.open(src).convert('RGB')         # verify it actually opens
            shutil.copy(src, os.path.join(dst_dir, f'{prefix}{n:05d}.jpg'))
            n += 1
        except Exception:
            pass                                   # skip corrupt files
    return n
```

3a. Download the Roboflow vape images (positives)

```
from roboflow import Roboflow

if not ROBOFLOW_API_KEY:
    raise ValueError('Please paste your ROBOFLOW_API_KEY in the config cell above.')

rf = Roboflow(api_key=ROBOFLOW_API_KEY)
project = rf.workspace(RF_WORKSPACE).project(RF_PROJECT)
version = project.version(RF_VERSION)

# download() returns an object whose .location is the extracted folder
try:
    rf_dataset = version.download(RF_EXPORT_FORMAT)
except Exception as e:
    print(f'"{RF_EXPORT_FORMAT}" export failed ({e}); retrying with "voc"...')
    rf_dataset = version.download('voc')

rf_path = rf_dataset.location
print('Roboflow dataset extracted to:', rf_path)

vape_files = collect_images(rf_path)
print('Found', len(vape_files), 'vape images in the download.')
n_vape = copy_sample(vape_files, VAPE_DIR, MAX_PER_CLASS)
print('Copied', n_vape, 'images into', VAPE_DIR)
```

```
loading Roboflow workspace...
loading Roboflow project...
"folder" export failed ({"error":"folder is an invalid format for project type object-detection. Please use
Downloading Dataset Version Zip in Vape-Dataset-1 to voc:: 100%|██████████| 45520/45520 [00:00<00:00, 58622

Extracting Dataset Version Zip to Vape-Dataset-1 in voc:: 100%|██████████| 3131/3131 [00:00<00:00, 8149.92i
Roboflow dataset extracted to: /content/Vape-Dataset-1
Found 1563 vape images in the download.
Copied 800 images into /content/data/vape
```

3b. Download Imagenette-320 everyday objects (base negatives)

```
# Imagenette-320: 10 everyday categories at >=320 px, from a stable public S3 URL.
# v2 uses the 320-px version (v1 used 160-px) so "low-res photo => not_vape" is no
# longer a shortcut the model can exploit.
IMAGENETTE_URL = 'https://s3.amazonaws.com/fast-ai-imageclas/imagenette2-320.tgz'

if len(collect_images(IMAGENETTE_DIR)) < 10:     # only download once
    !wget -q {IMAGENETTE_URL} -O /content/imagenette.tgz
    !tar -xzf /content/imagenette.tgz -C /content/
```

```
print('Imagenette-320 images available:', len(collect_images(IMAGENETTE_DIR)))
```

```
Imagenette-320 images available: 13394
```

3c. Download hard negatives — vape-lookalike objects (Open Images, no key needed)

These are the photos a lazy model confuses with vapes: elongated handheld gadgets. If the model separates *these* from vapes, it has learned actual vape features. We pull them from **Open Images** via `fiftyone` (keyless, only the matching images are downloaded — a few minutes on Colab). If the download fails for any reason, the notebook prints a warning and continues with Imagenette-only negatives (v1 behaviour).

```
HARD_CLASSES = ['Pen', 'Flashlight', 'Mobile phone', 'Remote control', 'Microphone']

if len(collect_images(HARD_NEG_DIR)) < 10:       # only download once
    try:
        import fiftyone.zoo as foz
        hard_ds = foz.load_zoo_dataset(
            'open-images-v7', split='validation',
            label_types=['classifications'], classes=HARD_CLASSES,
            max_samples=500, shuffle=True, seed=SEED,
            dataset_name='vape_hard_negatives')
        for i, fp in enumerate(hard_ds.values('filepath')):
            try:
                Image.open(fp).convert('RGB')     # skip anything unreadable
                shutil.copy(fp, os.path.join(HARD_NEG_DIR, f'hard_{i:05d}.jpg'))
            except Exception:
                pass
    except Exception as e:
        print('Hard-negative download failed — continuing with Imagenette-only negatives.')
        print('Reason:', e)

print('Hard-negative images available:', len(collect_images(HARD_NEG_DIR)))
```

```
invalid escape sequence '\Z'
Downloading split 'validation' to '/root/fiftyone/open-images-v7/validation' if necessary
INFO:fiftyone.zoo.datasets:Downloading split 'validation' to '/root/fiftyone/open-images-v7/validation' if n
Downloading 'https://storage.googleapis.com/openimages/2018_04/validation/validation-images-with-rotation.cs
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/2018_04/validation/va
Downloading 'https://storage.googleapis.com/openimages/v5/class-descriptions-boxable.csv' to '/root/fiftyon
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/v5/class-descriptions-
Downloading 'https://storage.googleapis.com/openimages/2018_04/bbox_labels_600_hierarchy.json' to '/tmp/tmpv
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/2018_04/bbox_labels_60
Downloading 'https://storage.googleapis.com/openimages/v5/validation-annotations-human-imagelabels-boxable.c
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/v5/validation-annotati
Only found 498 (<500) samples matching your requirements
WARNING:fiftyone.utils.openimages:Only found 498 (<500) samples matching your requirements
Downloading 498 images
INFO:fiftyone.utils.openimages:Downloading 498 images
 100% |███████████████████| 498/498 [47.9s elapsed, 0s remaining, 9.6 files/s]
INFO:eta.core.utils: 100% |███████████████████| 498/498 [47.9s elapsed, 0s remaining, 9.6 files/s]
Dataset info written to '/root/fiftyone/open-images-v7/info.json'
INFO:fiftyone.zoo.datasets:Dataset info written to '/root/fiftyone/open-images-v7/info.json'
Loading 'open-images-v7' split 'validation'
INFO:fiftyone.zoo.datasets:Loading 'open-images-v7' split 'validation'
 100% |███████████████████| 498/498 [1.5s elapsed, 0s remaining, 328.7 samples/s]
INFO:eta.core.utils: 100% |███████████████████| 498/498 [1.5s elapsed, 0s remaining, 328.7 samples/s]
Dataset 'vape_hard_negatives' created
INFO:fiftyone.zoo.datasets:Dataset 'vape_hard_negatives' created
Hard-negative images available: 498
```

3d. Compose the balanced `not_vape` folder (≈50% hard, ≈50% everyday)

We rebuild `not_vape/` from scratch on every run so the composition is deterministic (the copy order is driven by the seeded RNG). Hard negatives fill up to `HARD_NEG_FRACTION` of the folder; Imagenette fills the rest so the two classes stay balanced.

```
# Rebuild NOT_VAPE_DIR from scratch so re-runs are deterministic.
shutil.rmtree(NOT_VAPE_DIR, ignore_errors=True)
os.makedirs(NOT_VAPE_DIR, exist_ok=True)

n_vape = len(collect_images(VAPE_DIR))
n_hard_target = int(n_vape * HARD_NEG_FRACTION)

n_hard = copy_sample(collect_images(HARD_NEG_DIR), NOT_VAPE_DIR, n_hard_target, prefix='hard_')
```

```
n_easy = copy_sample(collect_images(IMAGENETTE_DIR), NOT_VAPE_DIR, n_vape - n_hard, prefix='easy_')

print(f'Negatives: {n_hard} hard (vape-lookalike) + {n_easy} everyday = {n_hard + n_easy}')
print('\nFinal counts:')
print('  vape    :', n_vape)
print('  not_vape:', len(collect_images(NOT_VAPE_DIR)))
```

```
Negatives: 400 hard (vape-lookalike) + 400 everyday = 800

Final counts:
  vape    : 800
  not_vape: 800
```

Step 4 — Build the dataset and the train / val / test split

`CLASS_NAMES = ['not_vape', 'vape']` so label **1 = vape** (the thing we want to catch).

We do a **stratified 70 / 15 / 15** split so all three sets keep the same vape:not-vape ratio. Splitting on the *file list* (before any transforms) guarantees the test images are never seen during training. The split is **seeded** (`SEED = 42` in Step 1), so every run — and both backbones — see exactly the same train/val/test files.

```
CLASS_NAMES = ['not_vape', 'vape']        # index 0 and 1
LABEL2NAME = {i: n for i, n in enumerate(CLASS_NAMES)}

# Build (path, label) pairs
all_samples = []
for label, name in enumerate(CLASS_NAMES):
    for f in collect_images(os.path.join(DATA_ROOT, name)):
        all_samples.append((f, label))
print('Total images:', len(all_samples))

# Stratified split: split each class separately, then merge
def split_indices(samples, train=0.70, val=0.15):
    by_class = {}
    for i, (_, lab) in enumerate(samples):
        by_class.setdefault(lab, []).append(i)
    tr, va, te = [], [], []
    for lab, idxs in by_class.items():
        random.shuffle(idxs)
        n = len(idxs); n_tr = int(n*train); n_va = int(n*val)
        tr += idxs[:n_tr]; va += idxs[n_tr:n_tr+n_va]; te += idxs[n_tr+n_va:]
    random.shuffle(tr); random.shuffle(va); random.shuffle(te)
    return tr, va, te

train_idx, val_idx, test_idx = split_indices(all_samples)
print(f'train {len(train_idx)} | val {len(val_idx)} | test {len(test_idx)}')

class VapeDataset(Dataset):
    # Returns (transformed_image, label). One dataset object per split/transform.
    def __init__(self, samples, indices, transform):
        self.items = [samples[i] for i in indices]
        self.transform = transform
    def __len__(self):
        return len(self.items)
    def __getitem__(self, idx):
        path, label = self.items[idx]
        img = Image.open(path).convert('RGB')
        return self.transform(img), label
```

```
Total images: 1600
train 1120 | val 240 | test 240
```

Step 5 — Data augmentation (and why)

Augmentation randomly perturbs each *training* image every epoch (flips, crops, colour jitter, rotation). The model therefore never sees the exact same picture twice, which:

- **reduces over-fitting** — important here because our dataset is small;
- **mimics real marketplace photos** — sellers shoot vapes at odd angles, distances, lighting and crops, so the training data should reflect that variety.

Validation and test images are *not* augmented — we only resize/centre-crop/normalise them, because at evaluation time we want a clean, deterministic measurement.

Two backbones expect **different normalisation**:

- **ResNet (ImageNet)** → mean `[0.485,0.456,0.406]`, std `[0.229,0.224,0.225]`
- **ViT (`google/vit-base-patch16-224`)** → the processor uses mean/std `[0.5,0.5,0.5]`

So we build a small dictionary of transforms per backbone.

```
IMG_SIZE = 224

NORM = {
    'resnet': dict(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    'vit':    dict(mean=[0.5, 0.5, 0.5],       std=[0.5, 0.5, 0.5]),
}

def make_transforms(backbone):
    m, s = NORM[backbone]['mean'], NORM[backbone]['std']
    train_t = transforms.Compose([
        transforms.Resize(256),
        transforms.RandomResizedCrop(IMG_SIZE, scale=(0.7, 1.0)),  # random zoom/crop
        transforms.RandomHorizontalFlip(),                         # left/right flip
        transforms.RandomRotation(15),                             # small tilt
        transforms.ColorJitter(0.2, 0.2, 0.2),                     # lighting/colour variation
        transforms.ToTensor(),
        transforms.Normalize(m, s),
    ])
    eval_t = transforms.Compose([                                  # clean, deterministic
        transforms.Resize(256),
        transforms.CenterCrop(IMG_SIZE),
        transforms.ToTensor(),
        transforms.Normalize(m, s),
    ])
    return train_t, eval_t

def make_loaders(backbone, batch_size=32):
    train_t, eval_t = make_transforms(backbone)
    ds_tr = VapeDataset(all_samples, train_idx, train_t)
    ds_va = VapeDataset(all_samples, val_idx,   eval_t)
    ds_te = VapeDataset(all_samples, test_idx,  eval_t)
    g = torch.Generator(); g.manual_seed(SEED)     # seeded shuffle order (v2)
    return {
        'train': DataLoader(ds_tr, batch_size=batch_size, shuffle=True,  num_workers=2,
                            generator=g, worker_init_fn=seed_worker),
        'val':   DataLoader(ds_va, batch_size=batch_size, shuffle=False, num_workers=2),
        'test':  DataLoader(ds_te, batch_size=batch_size, shuffle=False, num_workers=2),
    }
```

Visual check — what the augmented training images look like

Seeing the augmentation is the easiest way to confirm it is sensible (not so aggressive that the vape becomes unrecognisable).

```
train_t, _ = make_transforms('resnet')
inv_mean = np.array(NORM['resnet']['mean']); inv_std = np.array(NORM['resnet']['std'])

def show_tensor(ax, t):
    img = t.permute(1, 2, 0).numpy() * inv_std + inv_mean   # un-normalise for display
    ax.imshow(np.clip(img, 0, 1)); ax.axis('off')

sample_path, sample_label = all_samples[train_idx[0]]
fig, axes = plt.subplots(2, 4, figsize=(14, 7))
fig.suptitle(f'8 augmented versions of one "{LABEL2NAME[sample_label]}" image', fontsize=13)
for ax in axes.ravel():
    show_tensor(ax, train_t(Image.open(sample_path).convert('RGB')))
plt.tight_layout(); plt.show()
```



Step 6 — Build the two backbones with two strategies each

Both backbones are **pretrained on ImageNet**. Transfer learning means we keep those learned visual features and only re-teach the model our 2-class decision.

Head modification (required):

- **ResNet** ends in a fully-connected layer `fc` of size `(512 → 1000)`. We replace it with `(512 → 2)`.
- **ViT** ends in a `classifier` layer `(768 → 1000)`. We replace it with `(768 → 2)`.

The two strategies:

- `full` — every layer is trainable; the whole network adapts to vapes.
- `head` — we **freeze** the backbone (`requires_grad = False`) and train **only** the new head. This is faster, needs less data, and is the classic "feature-extractor" approach.

To keep one training loop for both libraries, we wrap the ViT so it returns plain **logits** (a tensor), exactly like the ResNet does.

```
NUM_CLASSES = 2
VIT_CHECKPOINT = 'google/vit-base-patch16-224'

class ViTWrapper(nn.Module):
    # Makes the HuggingFace ViT behave like a torchvision model: forward(x) -> logits tensor.
    def __init__(self, vit):
        super().__init__()
        self.vit = vit
    def forward(self, x):
        return self.vit(pixel_values=x).logits

def build_model(backbone, strategy):
    # backbone in {'resnet','vit'}; strategy in {'full','head'}.
    if backbone == 'resnet':
        net = resnet18(weights=ResNet18_Weights.DEFAULT)        # pretrained CNN
        net.fc = nn.Linear(net.fc.in_features, NUM_CLASSES)     # new 2-class head
        if strategy == 'head':
            for p in net.parameters(): p.requires_grad = False  # freeze backbone
            for p in net.fc.parameters(): p.requires_grad = True# unfreeze head
        return net.to(device)

    elif backbone == 'vit':
        vit = ViTForImageClassification.from_pretrained(
            VIT_CHECKPOINT, num_labels=NUM_CLASSES,
            ignore_mismatched_sizes=True)                       # lets us swap the head
        if strategy == 'head':
            for p in vit.vit.parameters(): p.requires_grad = False   # freeze encoder
            for p in vit.classifier.parameters(): p.requires_grad = True
        return ViTWrapper(vit).to(device)

    raise ValueError(backbone)
```

```
# Each backbone needs its own normalised data loaders (built once, reused).
LOADERS = {'resnet': make_loaders('resnet'), 'vit': make_loaders('vit')}
print('Loaders ready for:', list(LOADERS.keys()))
```

```
Loaders ready for: ['resnet', 'vit']
```

Step 7 — One training/evaluation loop for all runs

The loop is deliberately small and readable (same shape as Lab 3's). It:

- runs one epoch of train or eval,
- tracks loss + accuracy,
- keeps the **best weights by validation accuracy**,
- logs every epoch to **wandb** (if enabled).

We use **class weights** in the loss, computed from the *training split only*, so that if the two classes end up slightly unbalanced (e.g. the hard-negative download returns fewer images) the model still pays proportional attention to both. *(v1 claimed this but actually used an unweighted loss — fixed in v2.)*

```
def run_epoch(net, loader, criterion, optimizer=None):
    is_train = optimizer is not None
    net.train() if is_train else net.eval()
    total_loss, correct, total = 0.0, 0, 0
    with torch.set_grad_enabled(is_train):
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            logits = net(x)
            loss = criterion(logits, y)
            if is_train:
                optimizer.zero_grad(); loss.backward(); optimizer.step()
            total_loss += loss.item() * x.size(0)
            correct += (logits.argmax(1) == y).sum().item()
            total += x.size(0)
    return total_loss / total, correct / total

def train_config(backbone, strategy, epochs=5, lr=None):
    name = f'{backbone}_{strategy}'
    print(f'\n==================  {name}  ==================')
    net = build_model(backbone, strategy)
    loaders = LOADERS[backbone]

    # head-only can use a higher LR (only a small layer trains); full fine-tune uses a gentle LR.
    if lr is None:
        lr = 1e-3 if strategy == 'head' else 3e-5 if backbone == 'vit' else 1e-4
    params = [p for p in net.parameters() if p.requires_grad]
    optimizer = optim.AdamW(params, lr=lr)

    # Class weights from the TRAINING split only (never val/test): inverse-frequency
    # weighting so the rarer class contributes proportionally more to the loss.
    train_labels = np.array([all_samples[i][1] for i in train_idx])
    counts = np.bincount(train_labels, minlength=NUM_CLASSES)
    class_w = torch.tensor(counts.sum() / (NUM_CLASSES * counts),
                           dtype=torch.float32, device=device)
    print(f'  class weights (not_vape, vape): {class_w.cpu().numpy().round(3)}')
    criterion = nn.CrossEntropyLoss(weight=class_w)

    run = None
    if USE_WANDB:
        run = wandb.init(project=WANDB_PROJECT, name=name, reinit=True,
                         config=dict(backbone=backbone, strategy=strategy,
                                     lr=lr, epochs=epochs))

    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_acc, best_weights = 0.0, copy.deepcopy(net.state_dict())

    for epoch in range(epochs):
        tr_loss, tr_acc = run_epoch(net, loaders['train'], criterion, optimizer)
        va_loss, va_acc = run_epoch(net, loaders['val'],   criterion)
        history['train_loss'].append(tr_loss); history['val_loss'].append(va_loss)
        history['train_acc'].append(tr_acc);   history['val_acc'].append(va_acc)
        print(f'  epoch {epoch+1}/{epochs}  '
              f'train_loss {tr_loss:.3f} acc {tr_acc:.3f} | val_loss {va_loss:.3f} acc {va_acc:.3f}')
        if USE_WANDB:
            wandb.log({'epoch': epoch+1, 'train_loss': tr_loss, 'val_loss': va_loss,
```

```
                        'train_acc': tr_acc, 'val_acc': va_acc})
            if va_acc > best_acc:
                best_acc = va_acc; best_weights = copy.deepcopy(net.state_dict())

        net.load_state_dict(best_weights)          # restore best epoch
        print(f'  best val_acc = {best_acc:.3f}')
        if USE_WANDB and run is not None:
            wandb.summary['best_val_acc'] = best_acc
            run.finish()
        return net, history, best_acc
```

Step 8 — Train all four configurations

`EPOCHS` is small so the whole notebook finishes in a reasonable time on a Colab GPU. Increase it for better accuracy. Head-only runs are fast; full fine-tuning (especially the ViT) is the slow part.

```
EPOCHS = 5     # bump to 8–12 for stronger results if you have GPU time

CONFIGS = [('resnet', 'full'), ('resnet', 'head'),
           ('vit', 'full'),    ('vit', 'head')]

trained = {}     # name -> (net, history, best_val_acc)
for backbone, strategy in CONFIGS:
    net, hist, best = train_config(backbone, strategy, epochs=EPOCHS)
    trained[f'{backbone}_{strategy}'] = (net, hist, best)

print('\nDone. Trained configs:', list(trained.keys()))
```

```
==================  resnet_full  ==================
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpo
100%|██████████| 44.7M/44.7M [00:00<00:00, 119MB/s]
  class weights (not_vape, vape): [1. 1.]
  epoch 1/5  train_loss 0.164 acc 0.925 | val_loss 0.047 acc 0.983
  epoch 2/5  train_loss 0.038 acc 0.988 | val_loss 0.026 acc 0.992
  epoch 3/5  train_loss 0.042 acc 0.986 | val_loss 0.022 acc 0.992
  epoch 4/5  train_loss 0.022 acc 0.994 | val_loss 0.014 acc 0.996
  epoch 5/5  train_loss 0.014 acc 0.996 | val_loss 0.040 acc 0.988
  best val_acc = 0.996

==================  resnet_head  ==================
  class weights (not_vape, vape): [1. 1.]
  epoch 1/5  train_loss 0.433 acc 0.811 | val_loss 0.234 acc 0.929
  epoch 2/5  train_loss 0.221 acc 0.935 | val_loss 0.141 acc 0.958
  epoch 3/5  train_loss 0.177 acc 0.941 | val_loss 0.140 acc 0.938
  epoch 4/5  train_loss 0.148 acc 0.954 | val_loss 0.142 acc 0.938
  epoch 5/5  train_loss 0.143 acc 0.958 | val_loss 0.092 acc 0.967
  best val_acc = 0.967

==================  vit_full  ==================
config.json: 100%                                                 69.7k/69.7k [00:00<00:00, 5.71MB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher ra
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Pleas
[transformers] You passed `num_labels=2` which is incompatible to the `id2label` map of length `1000`.
model.safetensors: 100%                                           346M/346M [00:04<00:00, 2.67GB/s]
Loading weights: 100%                                             200/200 [00:00<00:00, 3916.63it/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key               | Status   |
------------------+----------+-------------------------------------------------------------------------------
classifier.bias   | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:torch.Size(
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs model:torch.

Notes:
- MISMATCH:    ckpt weights were loaded, but they did not match the original empty weight shapes.
  class weights (not_vape, vape): [1. 1.]
  epoch 1/5  train_loss 0.200 acc 0.929 | val_loss 0.068 acc 0.988
  epoch 2/5  train_loss 0.026 acc 0.996 | val_loss 0.043 acc 0.992
  epoch 3/5  train_loss 0.010 acc 0.999 | val_loss 0.022 acc 0.996
  epoch 4/5  train_loss 0.005 acc 0.999 | val_loss 0.015 acc 0.996
  epoch 5/5  train_loss 0.002 acc 1.000 | val_loss 0.017 acc 0.996
  best val_acc = 0.996

==================  vit_head  ==================
[transformers] You passed `num_labels=2` which is incompatible to the `id2label` map of length `1000`.
Loading weights: 100%                                             200/200 [00:00<00:00, 4014.51it/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key               | Status   |
------------------+----------+-------------------------------------------------------------------------------
classifier.bias   | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:torch.Size(
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs model:torch.

Notes:
- MISMATCH:    ckpt weights were loaded, but they did not match the original empty weight shapes.
  class weights (not_vape, vape): [1. 1.]
  epoch 1/5  train_loss 0.202 acc 0.933 | val_loss 0.084 acc 0.967
  epoch 2/5  train_loss 0.057 acc 0.984 | val_loss 0.052 acc 0.983
  epoch 3/5  train_loss 0.040 acc 0.990 | val_loss 0.051 acc 0.988
  epoch 4/5  train_loss 0.029 acc 0.995 | val_loss 0.035 acc 0.992
  epoch 5/5  train_loss 0.021 acc 0.998 | val_loss 0.034 acc 0.992
  best val_acc = 0.992

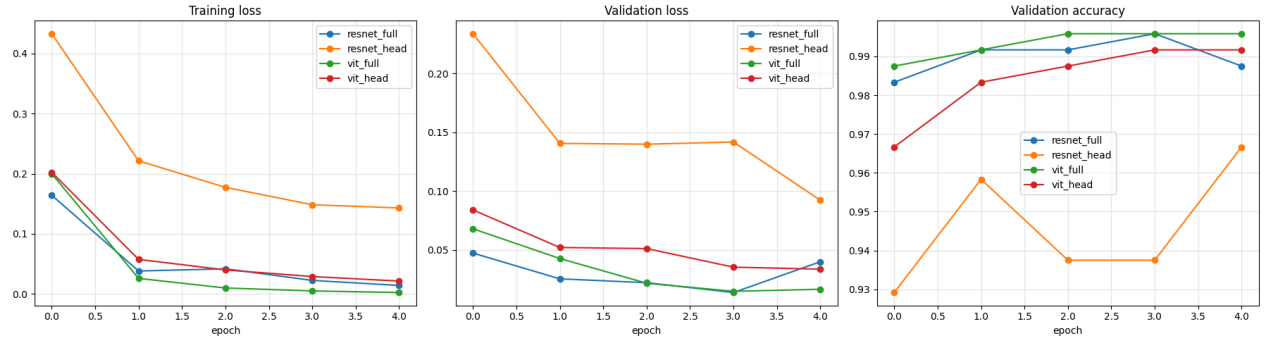
Done. Trained configs: ['resnet_full', 'resnet_head', 'vit_full', 'vit_head']
```

Step 9 — Compare the learning curves

Overlaying all four runs shows which configuration learns fastest and which generalises best (val accuracy is the honest signal; training accuracy can be misleadingly high).

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for name, (_, h, _) in trained.items():
    axes[0].plot(h['train_loss'], marker='o', label=name)
    axes[1].plot(h['val_loss'],   marker='o', label=name)
    axes[2].plot(h['val_acc'],    marker='o', label=name)
```

```
axes[0].set_title('Training loss')
axes[1].set_title('Validation loss')
axes[2].set_title('Validation accuracy')
for ax in axes:
    ax.set_xlabel('epoch'); ax.grid(True, alpha=.3); ax.legend()
plt.tight_layout(); plt.show()
```



Step 10 — Test-set evaluation: accuracy table + confusion matrices

The test set was held out from training. For a content-moderation tool we care about more than raw accuracy:

- **Recall (vape)** — of all real vapes, how many did we catch? *Misses = banned items slipping through.*
- **Precision (vape)** — of everything we flagged, how many were really vapes? *Low precision annoys honest sellers.*

So we report accuracy **and** precision/recall/F1, plus a confusion matrix per configuration.

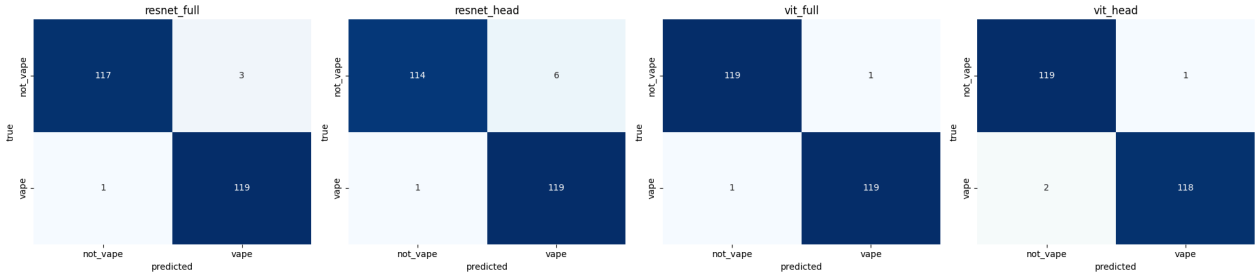
```
@torch.no_grad()
def predict_all(net, loader):
    net.eval()
    ys, ps = [], []
    for x, y in loader:
        logits = net(x.to(device))
        ps.append(logits.argmax(1).cpu().numpy()); ys.append(y.numpy())
    return np.concatenate(ys), np.concatenate(ps)

rows = []
cms = {}
for name, (net, _, best_val) in trained.items():
    backbone = name.split('_')[0]
    y_true, y_pred = predict_all(net, LOADERS[backbone]['test'])
    acc = accuracy_score(y_true, y_pred)
    pr, rc, f1, _ = precision_recall_fscore_support(
        y_true, y_pred, labels=[1], average='macro', zero_division=0)   # class 1 = vape
    rows.append((name, best_val, acc, pr, rc, f1))
    cms[name] = confusion_matrix(y_true, y_pred, labels=[0, 1])

# ----- accuracy table -----
print(f'{"config":16s} {"val_acc":>8s} {"test_acc":>9s} {"vape_P":>8s} {"vape_R":>8s} {"vape_F1":>8s}')
print('-'*62)
for name, bv, acc, pr, rc, f1 in rows:
    print(f'{name:16s} {bv:8.3f} {acc:9.3f} {pr:8.3f} {rc:8.3f} {f1:8.3f}')

# ----- confusion matrices -----
fig, axes = plt.subplots(1, 4, figsize=(20, 4.5))
for ax, (name, cm) in zip(axes, cms.items()):
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
                xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES, ax=ax)
    ax.set_title(name); ax.set_xlabel('predicted'); ax.set_ylabel('true')
plt.tight_layout(); plt.show()
```

```
config            val_acc  test_acc   vape_P   vape_R  vape_F1
------------------------------------------------------------
resnet_full         0.996     0.983    0.975    0.992    0.983
resnet_head         0.967     0.971    0.952    0.992    0.971
vit_full            0.996     0.992    0.992    0.992    0.992
vit_head            0.992     0.988    0.992    0.983    0.987
```



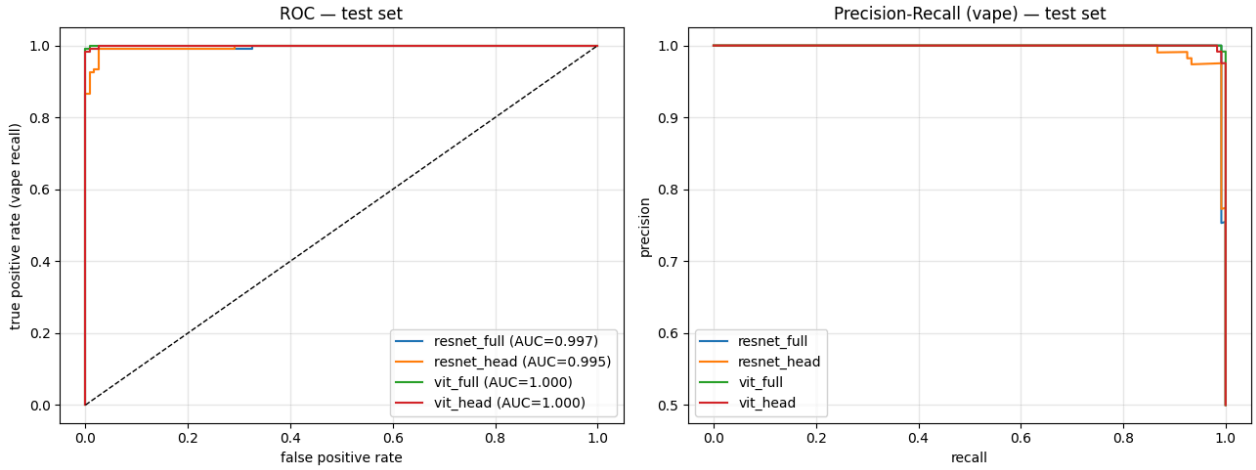
Step 10b — ROC / PR curves and a tuned moderation threshold *(new in v2)*

`argmax` predictions implicitly threshold the vape probability at **0.5**. A moderation product usually wants **high vape-recall** — catch (almost) every vape and let a human reviewer clear the extra false positives. So we:

1. plot **ROC** and **precision-recall** curves for all four configurations on the test set;
2. pick a **champion on validation metrics only** and **tune its threshold on the validation set** (tuning on the test set would be leakage): the threshold with the best precision whose validation vape-recall is still ≥ `TARGET_RECALL`;
3. report test-set metrics **once**, at 0.5 vs the tuned threshold;
4. look at the actual **misclassified test images** — each one is a talking point for the video.

```
@torch.no_grad()
def predict_probs(net, loader):
    # Returns (true labels, P(vape)) for every image in the loader, in loader order.
    net.eval()
    ys, ps = [], []
    for x, y in loader:
        p = F.softmax(net(x.to(device)), dim=1)[:, 1]
        ps.append(p.cpu().numpy()); ys.append(y.numpy())
    return np.concatenate(ys), np.concatenate(ps)

# ---- ROC + PR curves for all four configs on the TEST set (reporting only) ----
probs = {}                      # name -> (y_true, p_vape) on the test set
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
for name, (net, _, _) in trained.items():
    bb = name.split('_')[0]
    y_t, p_t = predict_probs(net, LOADERS[bb]['test'])
    probs[name] = (y_t, p_t)
    fpr, tpr, _ = roc_curve(y_t, p_t)
    prec_c, rec_c, _ = precision_recall_curve(y_t, p_t)
    axes[0].plot(fpr, tpr, label=f'{name} (AUC={auc(fpr, tpr):.3f})')
    axes[1].plot(rec_c, prec_c, label=name)
axes[0].plot([0, 1], [0, 1], 'k--', lw=1)
axes[0].set_title('ROC — test set'); axes[0].set_xlabel('false positive rate'); axes[0].set_ylabel('true p
axes[1].set_title('Precision-Recall (vape) — test set'); axes[1].set_xlabel('recall'); axes[1].set_ylabel(
for ax in axes: ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```




```
TARGET_RECALL = 0.99      # moderation policy: catch >= 99% of vapes on validation

def tune_threshold(y_true, p_vape, target_recall=TARGET_RECALL):
    # Lowest-FP operating point: among thresholds whose recall meets the target,
    # pick the one with the best precision. Falls back to 0.5 if none qualifies.
    best_th, best_prec = 0.5, -1.0
    for th in np.linspace(0.05, 0.95, 181):
        pred = (p_vape >= th).astype(int)
        if recall_score(y_true, pred, pos_label=1) >= target_recall:
            prec = precision_score(y_true, pred, pos_label=1, zero_division=0)
            if prec > best_prec:
                best_th, best_prec = th, prec
    return best_th, best_prec

# ---- validation metrics per config (also reused by Step 11 for selection) ----
val_stats = {}
for name, (net, _, _) in trained.items():
    bb = name.split('_')[0]
    y_v, p_v = predict_probs(net, LOADERS[bb]['val'])
    pred_v = (p_v >= 0.5).astype(int)
    val_stats[name] = dict(y=y_v, p=p_v,
                           acc=accuracy_score(y_v, pred_v),
                           rec=recall_score(y_v, pred_v, pos_label=1))

# champion = best validation accuracy, tie-broken by validation vape-recall
champion = max(val_stats, key=lambda n: (val_stats[n]['acc'], val_stats[n]['rec']))
print('Champion (chosen on validation only):', champion)

best_th, best_prec = tune_threshold(val_stats[champion]['y'], val_stats[champion]['p'])
print(f'Tuned threshold: {best_th:.2f}  '
      f'(validation precision {best_prec:.3f} at vape-recall >= {TARGET_RECALL})')

# ---- report on the TEST set, once, at 0.5 vs the tuned threshold ----
y_t, p_t = probs[champion]
print(f'\n{champion} on the TEST set:')
print(f'{"threshold":>10s} {"acc":>7s} {"vape_P":>8s} {"vape_R":>8s}   confusion [tn fp / fn tp]')
for th in [0.5, best_th]:
    pred = (p_t >= th).astype(int)
    cm = confusion_matrix(y_t, pred, labels=[0, 1])
    print(f'{th:10.2f} {accuracy_score(y_t, pred):7.3f} '
          f'{precision_score(y_t, pred, pos_label=1, zero_division=0):8.3f} '
          f'{recall_score(y_t, pred, pos_label=1):8.3f}   {cm.ravel().tolist()}')
```

```
Champion (chosen on validation only): resnet_full
Tuned threshold: 0.74  (validation precision 1.000 at vape-recall >= 0.99)

resnet_full on the TEST set:
 threshold     acc   vape_P   vape_R   confusion [tn fp / fn tp]
      0.50   0.983    0.975    0.992   [117, 3, 1, 119]
      0.74   0.996    1.000    0.992   [120, 0, 1, 119]
```

```
# ---- the actual mistakes: misclassified test images for the champion ----
# The test loader is not shuffled, so prediction order matches [all_samples[i] for i in test_idx].
test_items = [all_samples[i] for i in test_idx]
pred_t = (p_t >= best_th).astype(int)
wrong = np.where(pred_t != y_t)[0]
print(f'{len(wrong)} misclassified test images out of {len(y_t)} (threshold {best_th:.2f})')
```

```
if len(wrong):
    show = wrong[:8]
    fig, axes = plt.subplots(1, len(show), figsize=(3.2 * len(show), 3.8))
    for ax, i in zip(np.atleast_1d(axes), show):
        path, lab = test_items[i]
        ax.imshow(Image.open(path).convert('RGB')); ax.axis('off')
        ax.set_title(f'true: {LABEL2NAME[lab]}\npred: {LABEL2NAME[pred_t[i]]}  P(vape)={p_t[i]:.2f}',
                     fontsize=9)
    fig.suptitle(f'{champion} — every test-set mistake is a talking point', fontsize=12)
    plt.tight_layout(); plt.show()
else:
    print('No mistakes to show — consider raising HARD_NEG_FRACTION or adding more vape sources.')
```

```
1 misclassified test images out of 240 (threshold 0.74)
```



Step 11 — Select the two best configurations (on validation!) and save them

v2 fix: v1 ranked configurations by *test* accuracy — that quietly leaks the test set into model selection. We now rank by **validation accuracy**, tie-broken by **validation vape-recall** (the metric our moderation policy cares about), and the test set is only used for *reporting*.

Each checkpoint stores the weights, the metadata the web app needs (backbone, class names, normalisation) and the model's **tuned decision threshold** from Step 10b, so the demo apps can run at the same high-recall operating point.

```
# Rank on VALIDATION metrics only (val_stats was computed in Step 10b).
ranked = sorted(val_stats.items(), key=lambda kv: (kv[1]['acc'], kv[1]['rec']), reverse=True)
print('Validation ranking (accuracy, then vape-recall):')
for name, s in ranked:
    print(f"  {name:14s} val_acc {s['acc']:.3f} | val vape-recall {s['rec']:.3f}")

best_two = [name for name, _ in ranked[:2]]
print('\nTop-2 configurations (chosen WITHOUT looking at the test set):', best_two)

SAVE_DIR = '/content/best_models'
os.makedirs(SAVE_DIR, exist_ok=True)

manifest = []
for name in best_two:
    net = trained[name][0]
    backbone, strategy = name.split('_')
    # tune this model's own moderation threshold on the validation set
    th, th_prec = tune_threshold(val_stats[name]['y'], val_stats[name]['p'])
    ckpt_path = os.path.join(SAVE_DIR, f'{name}.pt')
    torch.save({
        'state_dict': net.state_dict(),
        'backbone': backbone,
        'strategy': strategy,
        'class_names': CLASS_NAMES,
        'img_size': IMG_SIZE,
        'norm': NORM[backbone],
        'vit_checkpoint': VIT_CHECKPOINT,
        'threshold': float(th),              # tuned on validation, recall >= TARGET_RECALL
    }, ckpt_path)
    test_acc = [r[2] for r in rows if r[0] == name][0]
    manifest.append({'name': name, 'path': ckpt_path,
                     'test_acc': float(test_acc), 'threshold': float(th)})
```

```
    print(f'saved {name}  ->  {ckpt_path}   (tuned threshold {th:.2f})')

import json
with open(os.path.join(SAVE_DIR, 'manifest.json'), 'w') as f:
    json.dump(manifest, f, indent=2)
print('\nWrote manifest.json. Download the best_models/ folder for the web-app deliverables.')
```

```
Validation ranking (accuracy, then vape-recall):
  resnet_full    val_acc 0.996 | val vape-recall 1.000
  vit_full       val_acc 0.996 | val vape-recall 1.000
  vit_head       val_acc 0.992 | val vape-recall 1.000
  resnet_head    val_acc 0.967 | val vape-recall 0.992

Top-2 configurations (chosen WITHOUT looking at the test set): ['resnet_full', 'vit_full']
saved resnet_full  ->  /content/best_models/resnet_full.pt   (tuned threshold 0.74)
saved vit_full  ->  /content/best_models/vit_full.pt   (tuned threshold 0.85)

Wrote manifest.json. Download the best_models/ folder for the web-app deliverables.
```

> **Tip:** On Colab, mount Google Drive and copy `best_models/` there so the web-app notebook can reload it without retraining:
>
> ```
> from google.colab import drive; drive.mount('/content/drive')
> !cp -r /content/best_models /content/drive/MyDrive/
> ```

```
from google.colab import drive; drive.mount('/content/drive')
!cp -r /content/best_models /content/drive/MyDrive/
```

Step 12 — Quick inference sanity check

A small helper that runs a single image through any saved model and returns class probabilities. The web-app deliverables reuse exactly this logic. Below we also launch a **minimal inline Gradio demo** so you can test the two best models without leaving the notebook (the polished seller-portal versions are separate deliverables).

```
import torch.nn.functional as F

def load_checkpoint(path):
    ckpt = torch.load(path, map_location=device)
    net = build_model(ckpt['backbone'], ckpt['strategy'])
    net.load_state_dict(ckpt['state_dict']); net.eval()
    eval_t = transforms.Compose([
        transforms.Resize(256), transforms.CenterCrop(ckpt['img_size']),
        transforms.ToTensor(), transforms.Normalize(ckpt['norm']['mean'], ckpt['norm']['std'])])
    return net, eval_t, ckpt['class_names']

@torch.no_grad()
def classify(pil_img, net, eval_t, class_names):
    x = eval_t(pil_img.convert('RGB')).unsqueeze(0).to(device)
    probs = F.softmax(net(x), dim=1)[0].cpu().numpy()
    return {class_names[i]: float(probs[i]) for i in range(len(class_names))}

# load the two best models
loaded = {m['name']: load_checkpoint(m['path']) for m in manifest}
print('Loaded for demo:', list(loaded.keys()))
```

```
[transformers] You passed `num_labels=2` which is incompatible to the `id2label` map of length `1000`.
Loading weights: 100%                                                    200/200 [00:00<00:00, 3640.51it/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key               | Status   |
------------------+----------+-------------------------------------------------------------------------------
classifier.bias   | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:torch.Size([1
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs model:torch.Si

Notes:
- MISMATCH:    ckpt weights were loaded, but they did not match the original empty weight shapes.
Loaded for demo: ['resnet_full', 'vit_full']
```

```
import gradio as gr

def compare(img):
```

```
        if img is None:
            return {}, {}
        outs = []
        for name in best_two:
            net, t, cn = loaded[name]
            outs.append(classify(img, net, t, cn))
        return outs[0], outs[1]

    with gr.Blocks(title='E-Vaporiser check (notebook demo)') as demo:
        gr.Markdown('### Quick two-model comparison\nUpload a listing photo to see both top models score it.'
        inp = gr.Image(type='pil', label='Listing photo')
        with gr.Row():
            out0 = gr.Label(num_top_classes=2, label=best_two[0])
            out1 = gr.Label(num_top_classes=2, label=best_two[1])
        inp.change(compare, inp, [out0, out1])
```